

Hansal Verrechnungsprogramm

Online-Hosting-Anleitung — die App dauerhaft & günstig im Internet betreiben

Das Ziel: Die App läuft *einmal* auf einem kleinen Server im Internet. Du und dein Freund öffnet einfach eine Web-Adresse im Browser – kein Kopieren von Dateien, kein Docker auf jedem PC, eine gemeinsame Datenbank für alle.

Warum ein einzelner kleiner Server (VPS) die beste Wahl ist

Die App ist bereits komplett "self-contained": Das Frontend ruft `/api` auf, und nginx leitet das intern an das Backend weiter. Die gesamte `docker-compose.yml` kann also **unverändert** auf einen Server gestellt und über Port 80 erreicht werden. Genau dasselbe Kommando wie lokal:

```
docker compose up -d --build
```

Deshalb ist ein kleiner VPS am einfachsten und am günstigsten – man muss die App nicht umbauen.

Kostenvergleich (Stand 2026, ungefähr)

Anbieter / Option	Spezifikation	Preis / Monat	Bemerkung
Hetzner Cloud CX22 EMPFEHLUNG	2 vCPU, 4 GB RAM, 40 GB SSD	~ €4–5	Deutsche Firma, EU/DSGVO, sehr stabil, reicht locker für diese App.
Hetzner CAX11 (ARM)	2 vCPU, 4 GB RAM, 40 GB SSD	~ €4	Noch etwas günstiger, ARM-Architektur (Docker läuft problemlos).
Oracle Cloud "Always Free"	bis 4 ARM-Kerne, 24 GB RAM	€0	Dauerhaft kostenlos, aber Anmeldung umständlich & Kapazität nicht garantiert.
Railway / Render / Fly.io	verwaltete Dienste	ab ~ €5–10	Bequem, aber App muss aufgeteilt werden + separate DB → teuer/komplexer.

Empfehlung: **Hetzner CX22** für ca. €4–5/Monat – zuverlässig und genug RAM, damit auch das Bauen (Maven + Angular) auf dem Server funktioniert. Wer 0 € will und etwas Geduld beim Einrichten mitbringt: Oracle Free Tier.

Einrichtung in 6 Schritten (Hetzner als Beispiel)

1. Server erstellen

- Konto bei `hetzner.com/cloud` anlegen, neues Projekt → "Server hinzufügen".
- Image: **Ubuntu 24.04**. Typ: **CX22** (4 GB RAM).
- SSH-Key hinterlegen (oder Passwort kommt per Mail). Server erstellen → du bekommst eine **IP-Adresse**.

2. Per SSH einloggen

```
# Vom eigenen PC aus (Mac/Linux Terminal oder Windows PowerShell):  
ssh root@DEINE_SERVER_IP
```

3. Docker installieren

```
curl -fsSL https://get.docker.com | sh
```

4. Projekt auf den Server holen

```
# Variante A – aus Git (empfohlen, später leicht aktualisierbar):  
git clone DEINE_REPO_URL hansal  
cd hansal  
  
# Variante B – vom eigenen PC hochladen (ohne Git):  
# scp -r ./hansal-verrechnungsprogramm-v3 root@DEINE_SERVER_IP:/root/hansal
```

5. Starten

```
docker compose up -d --build
```

Der erste Start dauert ein paar Minuten (Backend + Frontend werden gebaut).

6. Im Browser öffnen

```
http://DEINE_SERVER_IP
```

Fertig – die App ist online und für alle über diese Adresse erreichbar.

WICHTIG: Sicherheit vor dem öffentlichen Betrieb

Die mitgelieferte `docker-compose.yml` öffnet die Datenbank (Port 5432) und das Backend (Port 8080) fürs ganze Internet. Lokal egal – auf einem öffentlichen Server ein **Sicherheitsrisiko**. Vor dem Online-Betrieb unbedingt absichern:

- **DB/Backend-Ports nicht nach außen öffnen:** in `docker-compose.yml` die Ports auf localhost binden – aus `"5432:5432"` wird `"127.0.0.1:5432:5432"`, ebenso `"127.0.0.1:8080:8080"`. Nur das Frontend (Port 80) bleibt öffentlich.
- **Standard-Passwort ändern:** `POSTGRES_PASSWORD` in der `docker-compose.yml` durch ein eigenes, starkes Passwort ersetzen.
- **Firewall aktivieren** (nur SSH, HTTP, HTTPS erlauben):

```
ufw allow 22/tcp && ufw allow 80/tcp && ufw allow 443/tcp && ufw enable
```

Eigene Domain + HTTPS (optional, empfohlen)

Statt der nackten IP eine schöne Adresse wie `https://hansal.example.at` mit automatischem SSL-Zertifikat. Am einfachsten mit **Caddy** als vorgeschaltetem Webserver – A-Record der Domain auf die Server-

IP zeigen lassen, dann holt Caddy das Zertifikat selbst. Bei Bedarf liefere ich die fertige Caddy-Konfiguration nach.

Daten sichern (Backup)

Die Daten liegen im Docker-Volume `postgres_data` und überleben Neustarts. Regelmäßiges Backup in eine Datei:

```
docker exec hansal-postgres pg_dump -U hansal_user hansal_db > backup_$(date +%F).sql
```

Wiederherstellen:

```
docker exec -i hansal-postgres psql -U hansal_user hansal_db < backup_DATEI.sql
```

Niemals `docker compose down -v` verwenden – das *löscht* die Datenbank! Zum Stoppen reicht `docker compose down`.

App aktualisieren (neue Version einspielen)

```
cd hansal
docker compose down          # stoppt, Daten bleiben erhalten
git pull                     # neuen Code holen
docker compose up -d --build # neu bauen & starten
```

Nützliche Befehle (immer im Projektordner ausführen)

Zweck	Befehl
Alle Container (auch gestoppte) anzeigen	<code>docker compose ps -a</code>
Logs live ansehen	<code>docker compose logs -f</code>
Neustart	<code>docker compose restart</code>
Stoppen (Daten bleiben)	<code>docker compose down</code>
Läuft Docker?	<code>docker info</code>

Hansal Verrechnungsprogramm v3 · Backend: Spring Boot / Java 21 · Frontend: Angular 20 · Datenbank: PostgreSQL 16 · Betrieb über Docker Compose.